MANUAL DE DESARROLLO

SH-VT-TEC-001 | Versión 1.0

SECUREHUB / VULNTRACK

PLATAFORMA DE GESTIÓN DE VULNERABILIDADES Y RIESGOS

MANUAL DE DESARROLLO

Título del proyecto	SecureHub / VulnTrack
Subtitulo	**Plataforma de Gestión de Vulnerabilidades y Riesgos**
Documento	**Manual de Desarrollo**
Equipo de Ingeniería	**Muñoz Rey Hilary Michelle** *Analista de Documentación Técnica* **Ruiz Camelo Jennifer Dayan** *Especialista en Documentación de Software* **Molina Lancheros Daniel Steven** *Ingeniero DevSecOps* **Torres Pérez Eduar De Jesús** *Desarrollador de Software* **Alexander Diaz Molano** *Analista Técnico de Arquitectura*
Evaluador Técnico	Ospina Navas Jaider *Magíster en Ciberseguridad y Ciberdefensa*
Fecha de entrega	23 de octubre de 2026
Licenciamiento	MIT License (Código abierto) *Copyright © 2026 Equipo VulnTrack*
Versión del documento	**V1.0 — revisión de evidencias** *(10/10/2026)*

SECUREHUB · VULNTRACK

1. Objetivo

Establecer el procedimiento para que un integrante del equipo pueda obtener el código fuente de VulnTrack, preparar el entorno local, ejecutar el frontend y backend, correr las pruebas disponibles y proponer cambios de forma controlada.

2. Alcance y arquitectura de desarrollo

El proyecto contiene frontend React con Vite, API Node.js/Express, PostgreSQL, Redis, un worker, un escáner y servicios de monitoreo. El frontend puede ejecutarse en modo desarrollo con Vite en el puerto 5173; el despliegue completo está definido mediante Docker Compose.

Componente	Tecnología/configuración identificada	Uso
Frontend	React, Vite, Tailwind CSS; scripts dev/build/lint/preview	Interfaz de usuario
Backend	Node.js, Express; scripts dev/start/test	API y lógica de negocio
Pruebas backend	Jest + Supertest; carpeta backend/tests	Pruebas automatizadas del backend
Datos	PostgreSQL 18, Redis 7	Persistencia, caché y limitación de intentos
Calidad/CI	GitHub Actions, Gitleaks, Semgrep, Trivy, ZAP	Validaciones automatizadas definidas en workflows

3. Requisitos previos

- Windows 10/11 con PowerShell (procedimiento validado en la evidencia compartida); también se puede adaptar a otro sistema operativo.
- Git para clonar el repositorio.
- Docker Desktop con Docker Compose v2 y Docker Engine activo para ejecutar todos los servicios.
- Node.js y npm para ejecutar el frontend directamente en modo desarrollo. La imagen de contenedor usa Node.js 24; el workflow de pruebas backend usa Node.js 22.
- Acceso al repositorio https://github.com/3D74R08/VulnTrack.git.
- Puertos libres según docker-compose.yml: 80, 3000, 9090, 9093 y 3001, además de 5173 para Vite. PostgreSQL y Redis usan la red interna de Compose.

4. Obtener el código fuente

1. Abrir PowerShell.
2. Ubicarse en la carpeta en la que se guardará el proyecto.
3. Clonar el repositorio y entrar a la carpeta.

```
git clone https://github.com/3D74R08/VulnTrack.git
cd .\VulnTrack
git branch --show-current
git status
```

La evidencia de ejecución aportada muestra el clonado del repositorio y el uso de la rama main. Si el equipo trabaja en otra rama, confirmar el nombre con el mantenedor antes de iniciar cambios.

5. Configurar variables de entorno

```
Copy-Item .env.example .env
notepad .env
```

Editar el archivo .env local según los nombres de variables del .env.example y docker-compose.yml. Definir contraseñas fuertes y diferentes para la cuenta administradora de PostgreSQL, el usuario de aplicación, Redis y los secretos JWT. No copiar valores secretos al manual, a capturas ni al repositorio.

- No compartir ni versionar .env. Mantener .env.example con nombres y valores de ejemplo no sensibles.
- Usar DEMO_ADMIN_ENABLED=true únicamente cuando se necesite aprovisionar una cuenta de demostración en un entorno local nuevo; establecer DEMO_ADMIN_EMAIL y DEMO_ADMIN_PASSWORD con valores de prueba propios.
- NVD_API_KEY y variables SMTP pueden ser opcionales para ciertas funciones; su necesidad depende de la configuración de correo y enriquecimiento NVD.
- CORS_ORIGINS debe contener únicamente los orígenes necesarios para el entorno local.

6. Validar y levantar el entorno completo con Docker

1. Abrir Docker Desktop y esperar a que el motor indique que está listo.
2. Desde la raíz del repositorio, validar la sintaxis y resolución de variables.
3. Construir imágenes e iniciar los servicios en segundo plano.
4. Revisar el estado y los logs antes de acceder a la aplicación.

```
docker compose config --quiet
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

En la evidencia compartida, el comando docker compose up -d --build finalizó la construcción de las imágenes y docker compose ps mostró varios servicios en estado Up/healthy. Este resultado corresponde a esa ejecución; debe repetirse en cada equipo o versión.

7. Ejecutar el frontend en modo desarrollo

```
cd .\frontend
npm ci
npm.cmd run dev
```

Vite mostrará una dirección local, normalmente http://localhost:5173/. Mantener abierta la terminal mientras se utiliza el frontend. En PowerShell se puede usar npm.cmd para evitar conflictos con la política de ejecución de scripts.

8. Ejecutar el backend y las pruebas

El backend declara los scripts dev, start y test. Para instalar dependencias y correr pruebas localmente, usar:

```
cd .\backend
npm ci
npm test
```

Para ejecutar el servidor en modo desarrollo, usar npm run dev si las dependencias de desarrollo están instaladas. El backend usa variables de entorno y requiere que PostgreSQL y Redis estén accesibles con la configuración correspondiente.

```
npm run dev
```

La carpeta backend/tests contiene pruebas Jest. El workflow .github/workflows/tests.yml configura Node.js 22, ejecuta npm ci y npm test -- --ci. No se aportó un reporte de ejecución local de las pruebas, por lo que este manual documenta cómo correrlas, no afirma que todas hayan pasado.

9. Validaciones de calidad del frontend

```
cd .\frontend
npm ci
npm run lint
npm run build
```

Los scripts disponibles son dev, build, lint y preview. El build permite comprobar que la aplicación frontend se puede compilar. El lint ejecuta Oxlint según la configuración del proyecto.

10. Estrategia de ramas y contribuciones

La política exacta de ramas y aprobación debe acordarse por el equipo. Como práctica recomendada para el proyecto:

```
git switch main
git pull
 git switch -c feature/descripcion-corta
# realizar cambios y pruebas
```

```
git add .
git commit -m "feat: describe el cambio"
git push -u origin feature/descripcion-corta
```

Tipo de commit	Uso sugerido	Ejemplo
feat	Funcionalidad nueva	feat: agregar filtro de activos
fix	Corrección de error	fix: validar fecha límite
docs	Documentación	docs: actualizar manual de despliegue
test	Pruebas	test: cubrir middleware de permisos
refactor	Cambio interno sin alterar funcionalidad	refactor: separar servicio de riesgo
security	Cambio enfocado en seguridad	security: limitar origen CORS

1. Crear una rama descriptiva; no desarrollar directamente sobre main.
2. Ejecutar pruebas y análisis aplicables antes de abrir el Pull Request.
3. Describir objetivo, cambios, evidencia de pruebas y posibles riesgos en el Pull Request.
4. Solicitar revisión a otro integrante y resolver comentarios.
5. Integrar mediante el mecanismo acordado por el equipo y verificar los workflows de CI.

11. Problemas frecuentes

Síntoma	Acción recomendada
docker compose no reconoce el comando	Actualizar/iniciar Docker Desktop y verificar docker compose version.
Puerto ocupado	Identificar el proceso que utiliza el puerto o cambiar el mapeo documentado, validando referencias del frontend.
API no conecta a PostgreSQL	Revisar .env, docker compose ps y docker compose logs securehub-db securehub-api.
npm no se ejecuta en PowerShell	Usar npm.cmd; confirmar node --version y npm.cmd --version.
Pruebas fallan por servicios externos	Revisar configuración de prueba y logs; no marcar como exitosas hasta corregir y volver a ejecutar.